Control.Functor: https://hackage.haskell.org/package/base/docs/Data-Functor.html
Control.Applicative: https://hackage.haskell.org/package/base/docs/Control-Applicative.html
Control.Monad: https://hackage.haskell.org/package/base/docs/Control-Monad.html

Data.Functor

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$) :: a -> f b -> f a
($>) :: Functor f => f a -> b -> f b
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<&>) :: Functor f => f a -> (a -> b) -> f b
void :: Functor f => f a -> f ()
```

Control.Applicative

```
class Functor f => Applicative f where
  pure :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
  liftA2 :: (a -> b -> c) -> f a -> f b -> f c
  (*>) :: f a -> f b -> f b
  (<*) :: f a -> f b -> f a

class Applicative f => Alternative f where
   empty :: f a
  (<|>) :: f a -> f a -> f a
  some :: f a -> f [a]
  many :: f a -> f [a]

newtype Const a b = Const { getConst :: a }
newtype WrappedMonad m a = WrapMonad { unwrapMonad :: m a }
newtype WrappedArrow a b c = WrapArrow { unwrapArrow :: a b c }
newtype ZipList a = ZipList { getZipList :: [a] }

(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$) :: Functor f => a -> f b -> f a
(<**>) :: Applicative f => f a -> f (a -> b) -> f b
liftA :: Applicative f => (a -> b) -> f a -> f b
liftA3 :: Applicative f => (a -> b -> c -> d) -> f a -> f b -> f c
-> f d
optional :: Alternative f => f a -> f (Maybe a)
asum :: (Foldable t, Alternative f) => t (f a) -> f a
```

Control.Monad

```
-- Control.Monad
class Applicative m => Monad m where
    {-# MINIMAL pure, ((<*>) | liftA2) #-}
    (>>=)       :: forall a b. m a -> (a -> m b) -> m b
    (>>)        :: forall a b. m a -> m b -> m b
```

```
    return      :: a -> m a
    return      = pure

(=<<) :: Monad m => (a -> m b) -> m a -> m b
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
(<=<) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
(ap) :: Monad m => m (a -> b) -> m a -> m b

liftM :: Monad m => (a1 -> r) -> m a1 -> m r
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
liftM3 :: Monad m => (a1 -> a2 -> a3 -> r) -> m a1 -> m a2 -> m a3
-> m r
liftM4 :: ...
liftM5 :: ....


join :: Monad m => m (m a) -> m a
msum :: (Foldable t, MonadPlus m) => t (m a) -> m a
mfilter :: MonadPlus m => (a -> Bool) -> m a -> m a

filterM :: Applicative m => (a -> m Bool) -> [a] -> m [a]
mapAndUnzipM :: Applicative m => (a -> m (b, c)) -> [a] -> m ([b],
[c])
zipWithM :: Applicative m => (a -> b -> m c) -> [a] -> [b] -> m [c]
zipWithM_ :: Applicative m => (a -> b -> m c) -> [a] -> [b] -> m ()
foldM :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m
b
foldM_ :: (Foldable t, Monad m) => (b -> a -> m b) -> b -> t a -> m
()
replicateM :: Applicative m => Int -> m a -> m [a]
replicateM_ :: Applicative m => Int -> m a -> m ()
mapM :: (Traversable t, Monad m) => (a -> m b) -> t a -> m (t b)
mapM_ :: (Foldable t, Monad m) => (a -> m b) -> t a -> m ()
forM :: (Traversable t, Monad m) => t a -> (a -> m b) -> m (t b)
forM_ :: (Foldable t, Monad m) => t a -> (a -> m b) -> m ()
sequenceM :: (Traversable t, Monad m) => t (m b) -> m (t a)
sequenceM_ :: (Foldable t, Monad m) => t (m b) -> m ()
```

Do expression
Ref: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-470003.14

do
syntactic sugar for monad operations

```
-- do + <$>
(*2) <$> Just 1                     -- Just 2
do                                  -- do
 x <- Just 1
 return x*2

-- do + <*>
```

```
Just (*2) <*> Just 1                  -- Just 2
do
 f <- Just (*2)
 x <- Just 1
 return (f x)

-- do + >>
Just 1 >> Just 2                      -- Just 2
do                                    -- do
 Just 1
 Just 2

-- do + >>=
Just 1 >>= \x -> Just (2*x)           -- Just 2
do                                    -- do
 x <- Just 1
 Just (2*x)
```

do is also a generalization of **list comprehension**

```
-- list comprehension
[x+y |                      -- [12,14,16,18,20, ... 106,108,110]
  x <- [10,20..100]
  y <- [1..10]
  even x]

-- do
do
  x <- [10,20..100]
  y <- [1..10]
  guard (even x)
  return (x+y)
```

```
---- Basics
--
do x<-[1..10]; [1..x]      -- one line
do
 x<-[1..10]                -- multiline
 [1..x]

-- Each line must be monad
do
 x <- [1..10]
 [1,2,3]
 [1..x]
 ['a','b','c']   -- Change type
 return 1        -- [1]
 [1]             -- same
 1               -- ERROR. Not a monad
```

```
-- let expression
-- i.e. local variables
do
 let f = (*2)
 x <- [1..10]
 let z = [f x]
 z
```

```
Same
let f = (*2)
  in [1..10] >>=
    (\x->let z=[f x]
        in z)
```

```
-- e.g. (-> r)
do
 t <- (+10)       -- (+10) >>= (\t->(+(2*t))    i.e. 2(10+x)+x
 (+(2 * t))
 (+10)            -- ... >> (+10)
 t                -- ERROR. Must be a function

-- e.g. []
do
 x <- [1..10]     -- [1..10] >>= (\x->[1..x]) i.e.
 [1..x]
 [1,2,3]          --  ... >> [1,2,3]
 []               -- ... >> []
 x                -- ERROR. Must be a list

-- e.g. Maybe
do
 x <- Just 1      -- Just 1 >>= (\x->Just(10*x))   i.e. Just 2
 Just (10*x)
 Just 10          -- ... >> Just 10
 Nothing          -- ... >> Nothing
 2 * x            -- ERROR. Must be a Maybe
```

Applicative
Typeclassopedia: http://wiki.haskell.org/Typeclassopedia

```
---- Basics
--
-- Analogous to function application
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
mf::m (a->b), mx::m a
mf <*> mx            -- apply mx on mf i.e. mf(mx)
f::a->b, x:a
f x                  -- analogous in elements
```

```
-- e.g. application on a list
pure (+1) <*> [1..3]    -- [2,3,4]
(+1) 1                  -- 2. analogous in elements
(+1) 2
[(+1)] <*> [1..3]       -- same
(+1) <$> [1..3]         -- same


-- usage: function of multiple variables
mf <*> mx <*> my <*> mz     -- apply mx my mz on mf i.e.
mf(mx,my,mz)
                            -- where mf::m (a->b->c), mx::m a,
my::m b, mz::m c
f x y z                     -- analogous in elements
                            -- where f::a->b->c, x::a, y::b, z::c
((mf <*> mx) <*> my) <*> mz   -- same by left associativity

-- e.g. Applicative []
[(+)] <*> [1,10,100] <*> [1..3]       --
[2,3,4,11,12,13,101,102,103]
(+) 1 1                               -- analogous in elements
(+) 1 2
pure (+) <*> [1,10,100] <*> [1..3]    -- same
[(+1),(+10),(+100)] <*> [1..3]        -- same

-- partial application
[(+)] <*> [1,10,100]                  -- partially applied
(+) 1                                 -- analogous in elements
(+) 10
[(+1),(+10),(+100)]                   -- same


-- usage: composition aka chain
mh <*> (mg <*> (mf <*> mx))        -- composition i.e.
mh(mg(mf(mx))
                                   -- where mh::m (c->d), mg::m (b-
>c), mf::m (a->b), mx::m a
h (g (f x))                        -- analogous in base
                                   -- where h::c->d, g::b->c, f::a-
>b, x::a
mh <*> mg <*> mf <*> mx            -- ERROR <*> is left associative

-- e.g. Applicative []
pure (+1) <*> (pure (*10) <*> [1..3])    -- [11,21,31]
(+1) ((*10) 1)                           -- analogous in elements
[(+1)] <*> ([(*10)] <*> [1..3])          -- same
pure (+1) <*> [10,20,30]                 -- same


-- special case: composition with (.) operation
-- A special case of composition
[(.)] <*> [(+1)] <*> [(*10)] <*> [1..3]                --
[11,21,31] i.e. f(x)=10x+1
(.) (+1) (*10) 1                                       --
```

```
analogous in elements
pure (.) <*> pure (+1) <*> pure (*10) <*> [1..3]        -- same
[(.) (+1)] <*> [(*10)] <*> [1..3]
[(.) (+1) (*10)] <*> [1..3]

-- partially applied
[(.)] <*> [(+1)] <*> [(*10)]
pure (.) <*> pure (+1) <*> pure (*10)
(.) (+1) (*10)                                  -- analogous in
elements
[(.) (+1) (*10)]                                -- same
pure ((.) (+1) (*10))
[(.) (+1)] <*> [(*10)]


-- special case: map operation
-- A special case of map operation
f <$> mx
pure f <*> mx        -- same
-- e.g.  Applicative []
(+1) <$> [1..3]           -- [1,2,3]
pure (+1) <*> [1..3]      -- same
(+1) 1                    -- analogous in elements
(+1) 2
```

Utilities

```
(<**>) :: Applicative f => f a -> f (a -> b) -> f b
(*>) :: Applicative f => f a -> f b -> f b
(<*) :: Applicative f => f a -> f b -> f a

---- Applicative operators
--
-- <**>
<**>                         -- <*> flipped
(<**>) :: Applicative f => f a -> f (a -> b) -> f b
mx <**> my
my <*> mx                    -- same

-- *> <*
(*>) :: Applicative f => f a -> f b -> f b
*>                           -- ignore first
mx *> my                     -- my
pure const <*> mx <*> my
--
(<*) :: Applicative f => f a -> f b -> f a
mx <* my                              -- mx i.e. *> flipped
pure (flip const) <*> mx <*> my       -- same


-- <$>, <$: i.e. map related
--
(<$>) :: Functor f => (a -> b) -> f a -> f b
```

```
(<$>) = fmap
(<&>) :: Functor f => f a -> (a -> b) -> f b
f <$> mx
fmap f mx             -- same
f `fmap` mx
mx <&> f              -- same

-- multiple
f <$> g <$> mx        --
(f . g) <$> mx        -- same
mx <&> g <&> f        -- same


--
(<$) :: Functor f => a -> f b -> f a
<$                       -- <* analogue for values
x <$ my                  -- pure x
pure x <* my             -- same
my >> pure x
-- e.g.
1 <$ [1,2,3]            -- [1,1,1]
pure 1 <* [1,2,3]        -- same



---- Monad operators
--
-- (=<<): flip
(=<<) :: Monad m => (a -> m b) -> m a -> m b
=<< = flip (>>=)      -- flip
mx >>= k
k =<< mx              -- same

-- (>=>) (<=<):  aka Kielsli composition
-- composition for monad binds
(>=>) :: Monad m => (a -> m b) -> (b -> m c) -> a -> m c
(<=<) :: Monad m => (b -> m c) -> (a -> m b) -> a -> m c
x >>= (k >=> h)
x >>= (h <=< k)                      -- same
x >>= k >>= h                        -- same
x >>= (k >>= h)                      -- ERROR. Use Keisli
composition

-- e.g. composition
f :: a -> m d                    -- k :: a -> m b,  h :: b -> m c,
 j :: c -> m d
f x = x >>= k >>= h >>= j
f = k >=> h >=> j                -- same


-- (ap): same as (<*>) restricted to Monads
(ap) :: Monad m => m (a -> b) -> m a -> m b
(<*>) :: Applicative f => f (a -> b) -> f a -> f b
pure (*2) `ap` Just 2                  -- Just 4
```

```
pure (*2) <*> Just 2                    -- Just 4
pure (*) `ap` Just 2 `ap` Just 3        -- Just 6
pure (*) <*> Just 2 <*> Just 3          -- same


---- Lift* utils
--
-- i.e. a short hand for <*>
-- i.e. syntactic sugar to make <*> like normal function
application
--   liftA2 f mx my
--   liftM2 f mx my
--   f x y                -- analogous
--
-- liftA,liftM: unary
liftA :: Applicative f => (a -> b) -> f a -> f b
liftM :: Monad m => (a1 -> r) -> m a1 -> m r         -- same for
monads
liftA f mx
liftM f mx                  -- same
pure f <*> mx               -- same
f <$> mx                    -- same

-- e.g
liftA (*2) [1..3]          -- [2,4,6]
liftM (*2) [1..3]          -- same
pure (*2) <*> [1..3]       -- same
(*2) <$> [1..3]


-- liftA*, liftM*: n-ary
liftA2 :: Applicative f => (a -> b -> c) -> f a -> f b -> f c
liftM2 :: Monad m => (a1 -> a2 -> r) -> m a1 -> m a2 -> m r
liftA2 f mx my
liftM2 f mx my              -- same
f <>gt; mx <*> my           -- same
pure f <*> mx <*> my        -- same
...
liftA5
liftM5

-- e.g.
liftA2 (*) [1..3] [-1,10]        -- [-1,10,-2,20,-3,30]
liftM2 (*) [1..3] [-1,10]        -- same
pure (*) <*> [1..3] <*> [-1,10]  -- same


---- Monad utils
--
forever :: Applicative f => f a -> f b
forever

-- void: empty result
void :: Functor f => f a -> f ()
```

```
void = (>>(pure ()))   -- Dfn
void []                -- [()]
void $ Just 1          -- Just ()


---- Conditional:
--
-- guard: early exit
-- exit if False
guard :: Alternative f => Bool -> f ()
guard True = pure ()        -- Definition
guard False = empty
-- e.g.
evenOnly l = do
 x <- l
 guard (even x)
 return x
evenOnly [1..10]            -- [2,4,6,8]

-- when,unless: ?
when :: Applicative f => Bool -> f () -> f ()
unless :: Applicative f => Bool -> f () -> f ()
-- e.g.


-- join: remove one level
--
-- i.e. like concatenating
join :: Monad m => m (m a) -> m a
join = (>>=id)                    -- Dfn
join [[1,2],[3,4]]                -- [1,2,3,4]
concat [[1,2],[3,4]]              -- same
-- same as msum if MonadPlus
msum :: (Foldable t, MonadPlus m) => t (m a) -> m a
join [[1,2],[3,4]]
concat [[1,2],[3,4]]

--
mfilter :: MonadPlus m => (a -> Bool) -> m a -> m a
mfilter


---- *M Monad utils: Traversable analogues of list utils
-- e.g.
--  filter :: (a -> Bool) -> [a] -> [a]
--  filterM :: Applicative m => (a -> m Bool) -> [a] -> m [a]
--
-- Special cases of Traversable.traverse()
-- i.e. list [a] is a Traversable.
--
-- traverse + filter
filterM :: Applicative m => (a -> m Bool) -> [a] -> m [a]
filterM
filterM f
```

```
traverse (\x -> f x >> return x)        -- same
-- e.g.
evenFilter x = if even x then Just True else Nothing
filterM evenFilter [0]                -- Just [0]
filterM evenFilter [0,1]              -- Nothing
filterM evenFilter [0,2]              -- Just [0,2]

-- traverse + unzip
mapAndUnzipM :: Applicative m => (a -> m (b, c)) -> [a] -> m ([b],
[c])
mapAndUnzipM f x
unzip <$> traverse f x
mod3 x = Just (div x 3, mod x 3)
mapAndUnzipM mod3 [0..5]            -- Just ([0,0,0,1,1,1],
[0,1,2,0,1,2])

-- traverse + zip
zipWithM :: Applicative m => (a -> b -> m c) -> [a] -> [b] -> m [c]
zipWithM f x y
zipWithM_
traverse (\(x,y) -> f x y) (zip x y)
-- e.g.
sumZip x y = Just (x+y)
zipWithM sumZip [1,2,3] [4,5,6]               -- Just [5,7,9]

-- traverse + replicate
replicateM :: Applicative m => Int -> m a -> m [a]
replicateM n x
sequenceA (replicate n x)      -- same
replateteM_
-- e.g.
replicateM 3 (Just 1)          -- Just [1,1,1]
replicateM 3 Nothing           -- Nothing
```

MonadFail
Control.MonadFail: https://hackage.haskell.org/package/base/docs/Control-Monad-Fail.html

```
-- Control.MonadFail
class Monad m => MonadFail m where
  fail :: String -> m a
```

MonadFail
By default, **pattern matching** in a **do bind** is not supported because no behavior is defined if the match fails
MonadFail is a special class to allow pattern matching, by defining a **default value** for when a pattern fails.

e.g. Either a b
without MonadFail

```
f :: Num a => Either a [a] ->  Either a a
f x = do
  (v:_) <- x             -- ERROR. "No instance for (MonadFail
(Either a)) arising from a do statement with the failable pattern
'(v : vs)'"
  Right (2 * v)
```

with MonadFail

```
instance Num a => MonadFail (Either a) where
  fail _ = Left (-1)

f :: Num a => Either a [a] ->  Either a a
f x = do
  (v:_) <- x            -- OK due to MonadFail
  Right (2 * v)

-- use
f (Right [])            -- fail. Left (-1)
f (Right [1])           -- Right 2
```

==========

Applications

Alternative, MonadPlus
Control.Applicative: https://hackage.haskell.org/package/base/docs/Control-Applicative.html

```
-- Control.Applicative
class Applicative f => Alternative f where
    empty :: f a
    (<|>) :: f a -> f a -> f a
    some :: f a -> f [a]
    many :: f a -> f [a]


-- Control.Monad
class (Alternative m, Monad m) => MonadPlus m where
   mzero :: m a
   mplus :: m a -> m a -> m a

fold :: (Foldable f, Monoid t) => f t -> t
asum :: (Foldable f, Alternative t) => t (f a) -> f a
msum :: (Foldable f, MonadPlus t) => t (f a) -> f a
optional :: Alternative f => f a -> f (Maybe a)
```

Alternative, MonadPlus
Monoid for applicatives and monads.
Behave like either an **algebraic sum** (both paths x or y) or **OR** (only one of x or y) operation

e.g.

```
-- Maybe
Just 1 <|> Just 2                    -- Just 1
Nothing <|> Just 2                   -- Just 2

-- IO
putStrLn "a" <|> putStrLn "b"        -- "a"
fail "a" <|> putStrLn "b"            -- "b"

-- Parser
import Text.ParserCombinator.ReadP
p = char 'a' <|> char 'b'            -- parse 'a' OR 'b'
readP_to_S p "hello"                 -- fail []
readP_to_S p "ahello"                -- success [('a', "hello")]
readP_to_S p "bhello"                -- success [('b', "hello")]
p = string "a" <|> string "ah"       -- "a" OR "ah
readP_to_S p "ahello"                -- success both [("a",
"hello"),("ah", "hello")]
```

MonadPlus laws
Haskell wiki: https://wiki.haskell.org/MonadPlus
[SO] Distinction between MonadPlus, Alternative: https://stackoverflow.com/questions/10167879/distinction-between-typeclasses-monadplus-alternative-and-monoid

No consensus on interpretation. Either:
a) **algebraic sum x + y** (preferred): both paths x and y
e.g. Parser

```
-- List
[1,2,3] <|> [4,5,6]                                  --
[1,2,3,4,5,6]

-- Parser
import Text.ParserCombinator.ReadP
readP_to_S (string "a" <|> string "ah") "ah"         -- [("a",
"hello"), ("ah", "hello")]
```

b) **boolean x OR y**: y only if x fails
e.g.

```
-- IO
putStr "a" <|> putStr "b"              -- "a"
fail "" <|> putStr "b"                 -- "b"
```

```
-- Maybe
Just 1 <|> Just 2                          -- Just 1
Nothing <|> Just 2                         -- Just 2
```

Laws:

law 1	mzero >>= k	==	mzero	**zero law**
law 2a	(x `mplus` y) >>= k	==	(x >>= k) `mplus` (y >>= k)	**left distribution**

law for algebraic sum x + y. i.e. try both

OR

law 2b	(return a) `mplus` x	==	return a	**left catch**

law for boolean x OR y i.e. exit after first success

Alternative vs MonadPlus
Should be the same.
Q: Why are there separate classes then?
A: For historical reason
Ideally, only **Alternative** is needed since Applicative ⊃ Monad.
But Applicative and Alternative were added after Monad and MonadPlus.

Monoid vs Alternative/MonadPlus
[SO] Distinction between MonadPlus, Alternative: https://stackoverflow.com/questions/10167879/distinction-between-typeclasses-monadplus-alternative-and-monoid
In most cases, **monoid != alternative** (or monad plus)

Monoid instance	**instance Monoid a => Monoid (m a)** sum of a, not m itself
Alternative instance	**instance Alternative (m a)** algebraic sum of m

e.g. Monoid (Maybe a) != Alternative (Maybe a)

```
instance Monoid a => Monoid (Maybe a) where
  mempty = Nothing
  Just x <> Just y == Just (x <> y)
  Just x <> Nothing = Just x
  Nothing <> Just x = Just x
  Nothing <> Nothing = Nothing

instance Alternative (Maybe a) where
  empty = Nothing
  Just x <|> _ = Just x
  _ <|> y = y


-- e.g.
Just [1] <> Just [2]                -- Just [1,2]
Just [1] <> Nothing                 -- Just [1]
Just [1] <> Nothing <> Just [2]     -- Just [1,2]
-- ERROR
Just 1 <> Just 2                    -- ERROR. Int is not a Monoid
```

```
Just 1 <|> Just 2                    -- Just 1
Just 1 <|> Nothing                   -- Just 1
Nothing <|> Just 1                   -- Just 1
Nothing <|> Nothing                  -- Nothing
```

```
---- Utils
--
-- asum,msum: Foldable.fold for Alternatives,MonadPlus
-- i.e. for summing over a collection (Foldable)
--
fold :: (Foldable f, Monoid t) => f t -> t                -- Monoid
asum :: (Foldable f, Alternative t) => t (f a) -> f a     --
analogous for Alternative
msum :: (Foldable f, MonadPlus t) => t (f a) -> f a       --
analogous for MonadPlus
fold
foldr (<>) mempty                          -- same
asum
foldr (<|>) empty                          -- same
msum
foldr mplus mzero                          -- same

-- e.g.
asum [[1,2,3],[4,5,6]]                     -- [1,2,3,2,3]
msum [[1,2,3],[4,5,6]]                     -- all same
fold [[1,2,3],[4,5,6]]
concat [[1,2,3],[4,5,6]]


-- optional: 0 or 1
-- many: 0 or more
-- some: 1 or more
--
-- often used with parsers
optional :: Alternative f => f a -> f (Maybe a)
many :: Alternative f => f a -> f [a]
some :: Alternative f => f a -> f [a]
```

```
-- e.g. with a parser
import Control.Applicative
import Text.ParserCombinator.ReadP (char, readP_to_S)

parseA = char 'a'

-- optional
readP_to_S (optional parseA) "abc"        -- 0 or 1
 [(Nothing,"abc"),(Just 'a',"bc")]
readP_to_S (optional parseA) "bc"         -- 0 only
 [(Nothing,"bc")]
```

```
-- many: zero or more
readP_to_S (many parseA) "abc"            -- parse one
  [("a","bc")]
readP_to_S (many parseA) "aabc"           -- parse many
 [("a","aabc"),("aa","abc"),("aaa","bc")]
readP_to_S (many parseA) "bc"             -- parse none
 [("","bc")]

-- some: one or more
readP_to_S (some parseA) "aabc"           -- parse many
 [("a","aabc"),("aa","abc"),("aaa","bc")]
readP_to_S (some parseA) "bc"             -- parse none     [] i.e.
failed to parse
```